

Reddit-Style Discussion App Real-Time Notification System —

Production Design Date: 2025-08-10 UTC

Reddit-Style Discussion App Real-Time Notification System —

Production Design Date: 2025-08-10 UTC

One-Pager Summary

Goal Build a real-time, multi-tenant notification platform for a discussion app with sub-millisecond fanout inside a region and single-digit millisecond read latencies for online users. The system must be highly reliable, horizontally scalable, cost-aware, privacy-compliant, and developer-friendly.

Key Outcomes

Fresh, relevant notifications delivered within one second to online users and within a few seconds to mobile push and email. Strong correctness: no duplicates, ordered per user, per-device de-duplication, and guaranteed at-least-once to the user inbox. Fine-grained per-user preferences, quiet hours, rate limiting, and spam or abuse filtering. Multi-tenant isolation across communities or enterprises with per-tenant quotas and budgets. Operational excellence: clear SLOs, canary and automatic rollback, and rich observability.

Primary Assumptions

Tenants one to many thousands; users millions to hundreds of millions across regions. Peak write rate one hundred thousand to one million events per second globally, regionalized. Read QPS fifty thousand to five hundred thousand for websockets and API queries. Events include replies, mentions, post subscriptions, mod actions, upvotes, awards, chat pings, and system messages. Clients include web, iOS, Android, and email; push via APNs and FCM.

Functional Requirements

Capture events from posts, comments, votes, mentions, mod tools, chat, and system campaigns. Fanout notifications to targeted users with per-user ordering and de-duplication. Support top-level channels: in-app inbox, real-time websocket stream, mobile push, and email or digest. Preferences by type, community, severity, and channel. Quiet hours and do-not-disturb. Unread counts per scope such as global, per-community, and per-thread. Ack, mark-read, mark-unread, and hide or mute actions with sync across devices. Spam detection and throttling per actor and per recipient. Abuse escalation hooks. Admin tooling: replay, suppression, backfills, and campaign schedules. Tenant-level configuration and quotas. Data export and deletion APIs.

Non-Functional Requirements

Availability ninety nine point nine five percent for read or write APIs. Latency targets p95: websocket delivery under one hundred milliseconds in-region. Push dispatch under one second. Inbox fetch under ten milliseconds from cache. Durability RPO zero for accepted events. RTO minutes for regional failover. Scalability linear scale with partitions or shards. Regional cells with global control plane. Compliance SOC two Type two, GDPR, and CCPA with subject rights and retention policy.

Architecture Overview

High-Level Event producers write to regional ingestion gateways. A streaming backbone buffers and orders events. Stream processors perform routing, de-dupe, preference checks, spam filtering, and fanout to user inbox stores and delivery channels. Online users receive updates via websockets with millisecond latencies from an edge-aware gateway.

Ingestion

API Gateway terminates TLS, authenticates with OAuth or signed service tokens, validates

payloads, and rate-limits. Transactional Outbox or CDC from posting or commenting services publishes canonical events to Kafka topics keyed by tenant and target scope or user. Schema Registry manages versioned Protobuf or JSON schema. Dead-letter topics store failures with reason codes.

lag, processor throughput, redis ops per second,

p95 or p99 latencies, websocket fanout time, push success rate, token health, DLQ depth. Structured logs with tenant id, user id hash, request id, correlation id, and channel. SLO-aware autoscaling. Alerts on error budget burn, skewed partitions, hot keys, and cold caches. Runbooks for cache cold-start, shard failover, partition rebalancing, and regional read-only mode.

Capacity Planning and Cost Estimate Reference region with ten million daily active users and fifty thousand events per second sustained, peaking at two hundred thousand. Kafka six to nine brokers with one hundred to three hundred partitions per hot topic. Replication factor three. Flink parallelism one hundred twenty eight. Checkpoint interval five seconds. State sizes tuned for RocksDB on SSD. Redis cluster twelve to twenty four shards depending on payload size. Keep under sixty five percent memory utilization. DynamoDB or Scylla sized for two to five thousand writes per second per hot partition, with adaptive capacity or LWT for conflict avoidance. Rough monthly cost ballpark forty thousand to one hundred thousand per busy region depending on push volume and analytics scans.

Stream Processing

Flink or Kafka Streams performs idempotency, enrichment, preference evaluation, and routing. Per-user ordering by partitioning on user id hash within tenant. A small sliding window cache prevents duplicates. Spam filters evaluate actor velocity, IP reputation, and content risk flags. Fanout

strategy hybrid: immediate write to hot inbox for online recipients; batched for long-tail or digest channels. Serving Layer Hot inbox Redis Cluster stores per-user notification lists with capped lists for each scope. Sorted sets maintain last update timestamps. Hashes hold compact payloads. Unread counters kept in Redis with atomic increments and per-scope keys. Lua scripts provide multi-key atomicity for small updates. Persistent inbox store DynamoDB or Scylla stores durable notifications and read or hide state. Keys are tenant id and user id with range keys by time or sequence. Global Secondary Index enables type or community filters. Search or recall OpenSearch for keyword search within notifications or admin tooling. Email and push channels Delivery workers pull channel-eligible notifications and send via APNs, FCM, or ESP with exponential backoff and retry queues. Websocket gateway Edge-aligned gateway terminates persistent connections and subscribes to per-user event streams via pub or sub.

System of Record Authoritative event log: Kafka topics per tenant and region with adequate partitions, retention hours to days, and compaction for idempotency keys. Durable inbox table holds the immutable notification record, read state, and visibility flags. Redis is strictly a cache and counter store.

Data Flow Producer emits canonical event to Kafka through gateway. Processor enriches, dedupes, applies preferences, and routes to targets. Updates written to Redis inbox and counters, then persisted to DynamoDB or Scylla in batches. Websocket gateway publishes deltas to online users. Channel workers dispatch push or email asynchronously. Analytics sink captures anonymized events and deliveries to S3 in Parquet for cohort analysis.

Technology Stack Edge and auth Cloud CDN, API Gateway, WAF, OAuth two point one or OpenID Connect, mTLS for service to service. Streaming Kafka with Schema Registry and Kafka Connect. Alternative Kinesis for managed. Processing Apache Flink with RocksDB state backend and checkpoints to object storage. Hot cache Redis Cluster or KeyDB for lists, sets, hashes, and counters. Persistent store DynamoDB or Scylla or Cassandra. Postgres or Aurora for admin metadata and configs. Search OpenSearch. Analytics ClickHouse or BigQuery or Snowflake with Iceberg tables on S3 or GCS. Platform Kubernetes with gRPC between services. REST for public APIs. GraphQL for client aggregation. CI or CD GitHub Actions or GitLab CI, Argo CD, and Argo Rollouts.

Storage Strategy

Hot path Redis lists for per-scope recency, sorted sets for ranking by time and priority, hashes for payloads. TTL applied to long-tail items and cold scopes. Durable path DynamoDB or Scylla as the source of truth with time-ordered clustering, per-user partitions sized for hot users. Write amplification reduced via batching and conditional writes. Search path OpenSearch indexes selected fields for admin

queries and moderation tooling. Analytics path S3 or GCS in Parquet with hourly partitions and Iceberg table for efficient queries.

Scaling Approach Regional cell architecture. Each cell has its own Kafka, Flink, Redis, and persistent store. A thin global control plane handles auth, config, and routing. Kafka topics partitioned by tenant and user hash. Flink parallelism matches partition counts. Checkpoints under five seconds. Redis sharded with consistent hashing and hash tags per user key. Keys are compact and bounded to keep memory under

control. Websocket gateways autoscale on active connections and p95 latency. Sticky routing by user id to minimize cross-shard lookups. Cross-region replication optional for premium tenants. Read-only failover for websockets during regional incidents.

Data Models

Notification tenant id, user id, notif id, type, community id, thread id, actor id, verb, object id, created at, body json, priority, channels, visibility flags, read state, ttl UserPreferences tenant id, user id, defaults json, per type overrides, per community overrides, quiet hours tz aware, channel enablement, language InboxIndex hot tenant id, user id, scope key or list key, recent notif ids capped to n with timestamps UnreadCounter tenant id, user id, scope, count, updated at DeliveryAttempt notif id, channel, device id or address, status, attempt count, last error, last attempt at DeviceToken user id, platform, token or endpoint, app version, locale, last seen AdminEvent id, type replay or suppression or backfill, payload, who, when All tables include created and updated timestamps and audit fields. Keys include tenant id for multi-tenancy.

Latency Budgets and SLAs

Edge termination and auth under twenty milliseconds p95. Redis updates and reads under three milliseconds p95 within the same AZ. Websocket publish fanout under fifty milliseconds p95 after processor commit. Persistent write batching to DynamoDB or Scylla under five hundred milliseconds p95. Push dispatch to APNs or FCM under one second p95; email queuing under five seconds p95. Aggregate: event-to-online-user notification under one hundred milliseconds p95 in-region. Event-to-push under one second p95. Availability ninety nine point nine five percent for read and write APIs. Websocket connection availability ninety nine point nine percent per region.

Key Algorithms and Workflows

Fanout on write versus fanout on read Use a hybrid model. Online users get immediate writes to their hot inbox lists and counters. Cold users rely on durable store scans with on-demand caching. Idempotency and ordering Idempotency keys not just per event but per recipient. Processor emits exactly-once effective writes with conditional operations in Redis and durable store. Partitioning ensures per-user ordering. Preference evaluation Compile user preferences into a compact bitset or small JSON and cache in Redis. Evaluate early in the stream to reduce fanout. Deduplication and collapse Collapse bursts such as many upvotes on the same post into a single aggregated notification with counters and a sample of actors. Throttling and quiet hours Token buckets per recipient and per actor. Quiet hours deflect to queues or digests. A recurring job emits digests on schedule. Spam and abuse filters Heuristics and optional ML features on actor behavior. Quarantine suspicious notifications for moderator review or drop with scored reason codes. Delivery and retry Exponential backoff with jitter. Idempotent push or email senders. Circuit breakers per vendor. Feedback ingestion updates token health. Read synchronization Ack or mark-read updates Redis and durable store using a write-through with conflict detection. Websocket broadcasts read state to other devices.

Security, Reliability, and Compliance

PII minimization. Store only pseudonymous ids, not content bodies beyond a short cache. Encrypt in transit with TLS one point three and at rest with KMS, optionally per-tenant keys. Access control RBAC and ABAC. Scoped tokens. mTLS inside the mesh. Signed webhooks for partner or campaign sources. Strict CORS for public APIs. Privacy Data retention windows. Data subject access and deletion APIs. Regional data residency options for tenants. Reliability Multi-AZ for every tier. Kafka for backpressure and replay. DLQs and quarantine queues. Producer idempotency

and transactional outbox. Change management Canary releases, feature flags, and progressive traffic shifting with automatic rollback on SLO burn.

Multi-Tenancy

Tenant id mandated in every key, topic name, partitioning scheme, and metric. Per-tenant rate limits and budgets. Soft isolation by partitions and namespaces, optional hard isolation per large tenant with dedicated cells and VPCs.

CI or CD and Infrastructure as Code Terraform or Pulumi for clusters, topics, shards, IAM, WAF, and secrets. Helm for services and configs. Argo Rollouts for canary and rollback based on SLOs and error budgets. Database migrations use online DDL and dual-read windows. Contract tests enforce schema compatibility via the registry.

APIs and Integration Points POST v one events publish a canonical event from producers. GET v one inbox returns recent notifications for a scope with paging. GET v one counters returns unread counters by scope. POST v one ack mark read or unread or hide. GET v one preferences fetch effective preferences. PATCH v one preferences update rules. Websocket subscribe to user stream. Server emits deltas and counter updates. Admin APIs for replay, suppression, and campaign scheduling. Outbound integrations APNs, FCM, ESP, and optional Slack or webhooks for enterprise tenants.

Observability and Operational Practices OpenTelemetry everywhere. Tail-based sampling for websockets and stream processors. Golden dashboards

Testing Strategy and Failure Drills

Unit tests for routing, preference evaluation, dedupe, and counters. Property-based tests for ordering and idempotency. Contract tests for schemas and public APIs. Integration tests against APNs and FCM sandboxes. Load tests emphasize zset or list hot keys and websocket fanout under burst. Soak tests for memory fragmentation in Redis and RocksDB state growth. Chaos drills broker loss, Redis shard failover, websocket gateway churn, and regional failover to read-only. Game days rehearse digest backfills and large replay windows.

Phased Build Plan

MVP three months single region, single tenant, core event schema, Redis hot inbox, DynamoDB or Scylla durable store, websockets for real time, and basic push via APNs or FCM. Preferences and counters minimal. Phase two six to nine months multi-tenant, per-scope counters, spam and throttling, hybrid fanout, canary delivery, and analytics lake with ClickHouse dashboards. Phase three twelve months multi-region cells, advanced preference compiler, ML-assisted spam or ranking, per-tenant hard isolation, and stricter SLOs for premium tenants.

Key Choices, Trade-Offs, Risks, and Mitigations

Redis lists or zsets deliver the required latency but need disciplined memory hygiene. Keep payloads small and keys short, and cap lists. Exactly-once across services is impractical; use outbox, idempotency, and conditional writes for effective once semantics. Hybrid fanout balances cost and latency. Pure fanout-on-write can explode storage; pure fanout-on-read can miss strict SLOs for online users. Hot user partitions and celebrity actors create load skew. Mitigate with partitioning, caching, batching, and backpressure. Push providers have external variability. Use retries, circuit breakers, and multi-provider failover where needed.